Models and databases

3.2.10 Veritabanı erişimi optimizasyonu

Django'nun veritabanı katmanı, geliştiricilerin veritabanlarından en iyi şekilde yararlanmalarına yardımcı olacak çeşitli yollar sunar. Bu belge, ilgili belgelere bağlantıları bir araya getirir ve veritabanı kullanımınızı optimize etmeye çalışırken atmanız gereken adımları özetleyen bir dizi başlık altında düzenlenmiş çeşitli ipuçları ekler.

Önce profil oluşturun

Genel programlama uygulaması olarak, bu zaten aşikardır. Hangi sorguları yaptığınızı ve bunların size ne kadara mal olduğunu öğrenin. *QuerySet.explain()* işlevini kullanarak, belirli QuerySet'lerin veritabanınız tarafından nasıl yürütüldüğünü anlayın. Ayrıca, django-debug-toolbar gibi harici bir projeyi veya veritabanınızı doğrudan izleyen bir aracı kullanmak isteyebilirsiniz.

Gereksinimlerinize bağlı olarak, hız, bellek veya her ikisi için de optimizasyon yapabileceğinizi unutmayın. Bazen birini optimize etmek diğerine zarar verebilir, ancak bazen birbirlerine yardımcı olurlar. Ayrıca, veritabanı işlemi tarafından yapılan iş, Python işleminizde yapılan aynı miktardaki iş ile (sizin için) aynı maliyete sahip olmayabilir. Önceliklerinizin ne olacağına, dengenin nerede olması gerektiğine karar vermek ve tüm bunları gerektiği gibi profillemek size kalmıştır, çünkü bu, uygulamanıza ve sunucunuza bağlı olacaktır.

Aşağıda anlatılacak her şeyde, yaptığınız her değişiklikten sonra profil analizi yapmayı unutmayın; böylece, bu değişikliğin gerçekten bir fayda sağladığından ve kodunuzun okunabilirliğindeki azalmaya karşın yeterince büyük bir fayda sağladığından emin olabilirsiniz. Aşağıdaki önerilerin tümü, sizin durumunuzda genel ilkenin geçerli olmayabileceği, hatta tam tersi bir durumun söz konusu olabileceği ihtimalini göz önünde bulundurarak sunulmaktadır.

Standart veritabanı optimizasyon tekniklerini kullanın

. . .bunlar arasında şunlar yer alır:

- Dizinler. Profil oluşturma sürecinde hangi dizinlerin eklenmesi gerektiğini belirledikten sonra, bu en önemli önceliğiniz olmalıdır. Django'dan bunları eklemek için *Meta.indexes* veya *Field.db_index* kullanın. Dizinler aramaları hızlandırmaya yardımcı olabileceğinden, *filter()*, *exclude()*, *order_by()* vb. kullanarak sık sık sorguladığınız alanlara dizin eklemeyi düşünün. En iyi dizinleri belirlemek, özel uygulamanıza bağlı olan karmaşık ve veritabanına bağlı bir konudur. Bir dizini sürdürmenin getirdiği ek yük, sorgu hızındaki kazançlardan daha ağır basabilir.
- Alan türlerinin uygun kullanımı.

Yukarıda sıralanan adımları tamamladığınızı varsayacağız. Bu belgenin geri kalan kısmı, gereksiz iş yükü yaratmayacak şekilde Django'yu nasıl kullanacağınız üzerine odaklanmaktadır. Ayrıca bu belge, genel amaçlı önbellekleme gibi tüm kaynak tüketen işlemler için geçerli olan diğer optimizasyon tekniklerini ele almamaktadır.

QuerySet'leri Anlamak

QuerySet'leri anlamak, basit kodlarla iyi performans elde etmek için hayati önem taşır. Özellikle:

`QuerySet` değerlendirmesini anlamak

Performans sorunlarını önlemek için şunu anlamak önemlidir:

- QuerySet'lerin tembel olduğunu.
- Ne zaman değerlendirildiklerini.
- Verilerin bellekte nasıl tutulduğunu.

Önbelleğe alınmış öznitelikleri anlamak

`QuerySet`'in tamamının önbelleğe alınmasının yanı sıra, ORM nesnelerindeki öznitelikler sonuçları da önbelleğe alınır. Genel olarak, çağrılabilir olmayan öznitelikler önbelleğe alınır. Örneğin, şu örnek blog modellerini ele alalım:

```
>>> entry = Entry.objects.get(id=1)
>>> entry.blog    # Blog nesnesi bu noktada alınır
>>> entry.blog    # önbelleğe alınmış sürüm, DB erişimi yok
```

Ancak genel olarak, çağrılabilir öznitelikler her seferinde veritabanında arama yapılmasına neden olur:

```
>>> entry = Entry.objects.get(id=1)
>>> entry.authors.all()    # sorgu gerçekleştirildi
>>> entry.authors.all()    # sorgu tekrar gerçekleştirildi
```

Şablon kodunu okurken dikkatli olun - şablon sistemi parantez kullanımına izin vermez, ancak çağrılabilirleri otomatik olarak çağırır ve yukarıdaki ayrımı gizler.

Kendi özel özelliklerinize dikkat edin; gerektiğinde önbelleklemeyi (örneğin, *cached_property* dekoratörünü kullanarak) uygulamak size kalmıştır.

with şablon etiketini kullanın

`QuerySet`'in önbellekleme davranışından yararlanmak için *with* şablon etiketini kullanmanız gerekebilir.

iterator() işlevini kullanın

Çok sayıda nesneniz olduğunda, `QuerySet`'in önbellekleme davranışı büyük miktarda bellek kullanımına neden olabilir. Bu durumda, *iterator()* yardımcı olabilir.

explain() işlevini kullanın

QuerySet.explain(), kullanılan dizinler ve birleşimler dahil olmak üzere veritabanının bir sorguyu nasıl yürüttüğü hakkında ayrıntılı bilgi verir. Bu ayrıntılar, daha verimli bir şekilde yeniden yazılabilecek sorguları bulmanıza veya performansı artırmak için eklenebilecek dizinleri belirlemenize yardımcı olabilir.

Veritabanı işlemlerini Python yerine veritabanında gerçekleştirin

Örneğin:

- En temel düzeyde, veritabanında filtreleme yapmak için filter ve exclude kullanın.
- Aynı model içindeki diğer alanlara göre filtreleme yapmak için *F ifadeleri* kullanın.
- Veritabanında toplama işlemi yapmak için annotate

kullanın. Bunlar ihtiyacınız olan SQL'i oluşturmak için

yeterli değilse:

RawSQL'i kullanın

Daha az taşınabilir ancak daha güçlü bir yöntem olan *RawSQL* ifadesi, sorguya bazı SQL kodlarının açıkça eklenmesine olanak tanır. Bu da hala yeterince güçlü değilse:

Raw SQL'i kullanın

Verileri almak veya modelleri doldurmak için kendi özel SQL'inizi yazın. Django'nun sizin için ne yazdığını öğrenmek için `django.db.connection.queries`'i kullanın ve oradan başlayın.

Benzersiz, indeksli bir sütun kullanarak tek tek nesneleri alma

get() işlevini kullanarak tek tek nesneleri almak için *unique* veya *db_index* ile bir sütun kullanmanın iki nedeni vardır. İlk olarak, altta yatan veritabanı dizini sayesinde sorgu daha hızlı olacaktır. Ayrıca, aramaya birden fazla nesne eşleşirse sorgu çok daha yavaş çalışabilir; sütunda unique kısıtlaması olması, bunun asla olmayacağını garanti eder.

Örnek blog modellerini kullanarak:

```
>>> entry = Entry.objects.get(id=10)
```

şundan daha hızlı olacaktır:

```
>>> entry = Entry.objects.get(headline="Haber Başlığı")
```

çünkü `id`, veritabanında indekslenmiştir ve benzersiz olduğu garanti edilir.

Aşağıdakini yapmak potansiyel olarak oldukça yavaş olabilir:

```
>>> entry = Entry.objects.get(headline__startswith="News")
```

Öncelikle, `başlık` indekslenmemiştir, bu da altta yatan veritabanının daha yavaş veri almasına neden olacaktır.

İkincisi, bu arama işleminin yalnızca tek bir nesnenin döndürüleceğini garanti etmez. Sorgu birden fazla nesneyle eşleşirse, veritabanından tümünü alır ve aktarır. Yüzlerce veya binlerce kayıt döndürülürse bu performans kaybı oldukça büyük olabilir. Veritabanı ayrı bir sunucuda bulunuyorsa, ağ yükü ve gecikme süresinin de etkisiyle bu performans kaybı daha da artacaktır.

İhtiyacınız olacağını biliyorsanız her şeyi tek seferde alın

Tüm parçalarına ihtiyacınız olacak tek bir veri "kümesinin" farklı parçaları için veritabanına birden fazla kez başvurmak, genel olarak hepsini tek bir sorguda almak kadar verimli değildir. Bu, döngü içinde yürütülen ve bu nedenle tek bir sorgu yeterliyken birçok veritabanı sorgusu yapmanıza neden olabilecek bir sorgunuz varsa özellikle önemlidir. Öyleyse:

QuerySet.select_related() ve prefetch_related() işlevlerini kullanın

select_related() ve *prefetch_related()* işlevlerini iyice anlayın ve kullanın:

- uygun olduğu durumlarda yöneticilerde ve varsayılan yöneticilerde. Yöneticinizin ne zaman kullanıldığını ve ne zaman kullanılmadığını iyi bilin; bu bazen kafa karıştırıcı olabilir, bu yüzden varsayımlarda bulunmayın.
- görünüm kodunda veya diğer katmanlarda, gerektiğinde *prefetch_related_objects()* işlevini kullanarak.

İhtiyacınız olmayan şeyleri almayın

QuerySet.values() ve values_list() işlevlerini kullanın

Yalnızca bir `sözlük` veya değerler `listesi` istediğinizde ve ORM model nesnelerine ihtiyacınız olmadığında, *values()* işlevini uygun şekilde kullanın. Bu, şablon kodunda model nesnelerinin yerine geçirmek için yararlı olabilir; sağladığınız sözlükler şablonda kullanılanlarla aynı özniteliklere sahip olduğu sürece sorun yoktur.

QuerySet.defer() ve only() kullanın

Yüklemelerini önlemek için, ihtiyaç duymayacağınızı (veya çoğu durumda ihtiyaç duymayacağınızı) bildiğiniz veritabanı sütunları varsa *defer()* ve *only()* işlevlerini kullanın. Bunları kullanırsanız, ORM'nin bunları ayrı bir sorguyla alması gerekeceğini ve bu işlevleri uygunsuz bir şekilde kullanmanız durumunda bunun bir performans düşüşüne yol açacağını unutmayın.

Performans analizi yapmadan alanları ertelemekte fazla agresif davranmayın; çünkü veritabanı, sonuçlardaki tek bir satır için metin ve VARCHAR olmayan verilerin çoğunu diskten okumak zorunda kalır; sonuçta sadece birkaç sütun kullanılmış olsa bile. `defer()` ve `only()` yöntemleri, çok fazla metin verisinin yüklenmesini önleyebildiğiniz durumlarda veya Python'a geri dönüştürülmesi çok fazla işlem gerektirebilecek alanlar için en yararlıdır. Her zamanki gibi, önce performans analizi yapın, sonra optimize edin.

QuerySet.contains(obj) kullanın

. . .eğer sadece `obj'nin` queryset'te olup olmadığını öğrenmek istiyorsanız, "`obj in queryset`" yerine.

QuerySet.count() kullanın

. . .`len(queryset)` yapmak yerine sadece sayıyı öğrenmek istiyorsanız.

QuerySet.exists() kullanın

. . .`queryset'te olup olmadığını` değil, en az bir sonuç olup olmadığını öğrenmek istiyorsanız. Ancak:

contains(), count() ve exists() işlevlerini aşırı kullanmayın

QuerySet'ten başka verilere ihtiyacınız olacaksa, bunları hemen değerlendirin.

Örneğin, `User` ile çok-çok ilişkisi olan bir `Group` modeli olduğunu varsayarsak, aşağıdaki kod en uygunudur:

```
members = group.members.all()

if display_group_members:
    eğer üyeler varsa:
        if current_user in members:
            print("Siz ve", len(members) - 1, "diğer kullanıcılar bu grubun üyesisiniz.")
        else:
            print("Bu grupta", len(members), "üye var.")

        members listesindeki her üye
            için: member.username'i yazdır
    else:
        print("Bu grupta üye yok.")
```

Bu kodun en uygun olması nedeni:

1. QuerySet'ler tembel işleme özelliğine sahip olduğundan, `display_group_members` değeri `False` ise bu işlem herhangi bir veritabanı sorgusu gerçekleştirmez.
2. `group.members.all()` ifadesini `members` değişkeninde saklamak, sonuç önbelleğinin yeniden kullanılmasını sağlar.
3. `if members:` satırı, `QuerySet.bool()`'un çağrılmasına neden olur; bu da `group.members.all()` sorgusunun veritabanında çalıştırılmasına yol açar. Herhangi bir sonuç yoksa `False`, aksi takdirde `True` döndürür.
4. "`if current_user in members:`" satırı, kullanıcının sonuç önbelleğinde olup olmadığını kontrol eder; böylece ek veritabanı sorguları gönderilmez.
5. `len(members)` kullanımı, `QuerySet.__len__()` işlevini çağırır ve sonuç önbelleğini yeniden kullanır, bu nedenle yine veritabanı sorgusu gönderilmez.
6. `for member` döngüsü, sonuç önbelleği üzerinde yinelenir.

Toplamda, bu kod bir veya sıfır veritabanı sorgusu gerçekleştirir. Yapılan tek kasıtlı optimizasyon, `members` değişkeninin kullanılmasıdır. `if` için `QuerySet.exists()`, `in` için `QuerySet.contains()` veya count için `QuerySet.count()` kullanılması, her biri ek sorgulara neden olur.

QuerySet.update() ve delete() kullanın

Çok sayıda nesneyi tek tek alıp bazı değerleri ayarladıktan sonra tek tek kaydetmek yerine, QuerySet.update() aracılığıyla toplu SQL UPDATE deyimi kullanın. Benzer şekilde, mümkün olduğunda toplu silme işlemleri de gerçekleştirin.

Ancak, bu toplu güncelleme yöntemlerinin tek tek örneklerin `save()` veya `delete()` yöntemlerini çağıramayacağını unutmayın; bu, normal veritabanı nesnesi sinyallerinden kaynaklananlar da dahil olmak üzere, bu yöntemler için eklediğiniz herhangi bir özel davranışın yürütülmeyeceği anlamına gelir.

Yabancı anahtar değerlerini doğrudan kullanın

Yalnızca bir yabancı anahtar değerine ihtiyacınız varsa, ilgili nesnenin tamamını alıp birincil anahtarını almak yerine, elinizdeki nesnede zaten bulunan yabancı anahtar değerini kullanın. Yani şunu yapın:

```
entry.blog_id
```

yerine şunu yapın:

```
entry.blog.id
```

Önemli değilse sonuçları sıralama

Sıralama işlemi ücretsiz değildir; sıralama yapılacak her alan, veritabanının gerçekleştirmesi gereken bir işlemdir. Bir modelin varsayılan sıralaması (*Meta.ordering*) varsa ve buna ihtiyacınız yoksa, `QuerySet` üzerinde parametre vermeden *order_by() işlevini* çağırarak bunu kaldırın.

Veritabanınıza bir dizin eklemek, sıralama performansını artırmaya yardımcı olabilir.

Toplu yöntemleri kullanın

SQL deyimlerinin sayısını azaltmak için toplu yöntemleri kullanın.

Toplu olarak oluşturun

Nesneler oluştururken, mümkün olduğunda SQL sorgu sayısını azaltmak için *bulk_create()* yöntemini kullanın. Örneğin:

```
Entry.objects.bulk_create( [
        Entry(headline="Bu bir test"), Entry(headline="Bu sadece
        bir test"),
    ]
)
```

. . .şu koddan daha tercih edilebilir:

```
Entry.objects.create(headline="Bu bir test") Entry.objects.create(headline="Bu sadece
bir test")
```

Bu yöntemin bazı *uyarıları* olduğunu unutmayın, bu nedenle kullanım durumunuza uygun olduğundan emin olun.

Toplu güncelleme

Nesneleri güncellerken, mümkünse SQL sorgu sayısını azaltmak için *bulk_update()* yöntemini kullanın. Bir nesne listesi veya sorgu kümesi verildiğinde:

```
entries = Entry.objects.bulk_create( [
        Entry(headline="Bu bir test"), Entry(headline="Bu sadece
        bir test"),
    ]
)
```

Aşağıdaki örnek:

```
girişler[0].başlık = "Bu bir test değildir" girişler[1].başlık =
"Bu artık bir test değildir" Entry.objects.bulk_update(girişler,
["başlık"])
```

. . .şu koddan daha tercih edilebilir:

```
entries[0].headline = "Bu bir test değildir"
entries[0].save()
entries[1].headline = "Bu artık bir test değil"
entries[1].save()
```

Bu yöntemin bir takım *uyarıları* olduğunu unutmayın, bu nedenle kullanım durumunuza uygun olduğundan emin olun.

Toplu ekleme

ManyToManyField'lara nesneler eklerken, SQL sorgu sayısını azaltmak için *add() yöntemini* birden fazla nesneyle birlikte kullanın. Örneğin:

```
my_band.members.add(me, my_friend)
```

. . .şu şekilde yapmaktan daha iyidir:

```
my_band.members.add(me)
my_band.members.add(my_friend)
```

. . .burada `Band` ve `Artist`, çok-çok ilişkisine sahip modellerdir.

ManyToManyField'a farklı nesne çiftleri eklerken veya özel *geçiş* tablosu tanımlandığında, SQL sorgu sayısını azaltmak için *bulk_create()* yöntemini kullanın. Örneğin:

```
PizzaToppingRelationship = Pizza.toppings.through PizzaToppingRelationship.objects.bulk_create(
    [
        PizzaToppingRelationship(pizza=my_pizza, topping=pepperoni),
        PizzaToppingRelationship(pizza=your_pizza, topping=pepperoni),
        PizzaToppingRelationship(pizza=your_pizza, topping=mushroom),
    ],
    ignore_conflicts=True,
)
```

. . .şu koddan daha tercih edilebilir:

```
my_pizza.toppings.add(pepperoni)
your_pizza.toppings.add(pepperoni, mushroom)
```

. . .burada `Pizza` ve `Topping` *arasında* çok-çok ilişkisi vardır. *Bu yöntemin* bir dizi *uyarı noktası* olduğunu unutmayın, bu nedenle kullanım durumunuza uygun olduğundan emin olun.

Toplu olarak silme

ManyToManyFields'tan nesneleri silerken, SQL sorgu sayısını azaltmak için *remove() işlevini* birden fazla nesneyle birlikte kullanın. Örneğin:

```
my_band.members.remove(me, my_friend)
```

. . .şu şekilde yapmaktan daha iyidir:

```
my_band.members.remove(me)
my_band.members.remove(my_friend)
```

. . .burada `Band` ve `Artist`, çok-çok ilişkisine sahip modellerdir.

ManyToManyFields'tan farklı nesne çiftlerini kaldırırken, SQL sorgu sayısını azaltmak için birden *fazla* model örneği *içeren* bir *Q* ifadesinde *delete()* işlevini kullanın. Örneğin:

```
from django.db.models import Q

PizzaToppingRelationship = Pizza.toppings.through PizzaToppingRelationship.objects.filter(
    Q(pizza=my_pizza, topping=pepperoni)
    | Q(pizza=senin_pizzan, topping=pepperoni)
    | Q(pizza=senin_pizzan,  topping=mantar)
).delete()
```

. . .şu koddan daha tercih edilebilir:

```
my_pizza.toppings.remove(pepperoni)
your_pizza.toppings.remove(pepperoni, mushroom)
```

. . .burada `Pizza` ve `Topping` arasında çok-çok ilişkisi vardır.

3.2.11 Veritabanı enstrümantasyonu

Kodunuz tarafından gönderilen sorguları anlamanıza ve kontrol etmenize yardımcı olmak için Django, veritabanı sorgularının yürütülmesi sürecine sarmalayıcı işlevler eklemenizi sağlayan bir kanca sunar. Örneğin, sarmalayıcılar sorgu sayısını sayabilir, sorgu süresini ölçebilir, sorguları günlüğe kaydedebilir veya hatta sorgu yürütülmesini engelleyebilir (örneğin, önceden alınmış verilerle bir şablon işlenirken hiçbir sorgu gönderilmediğinden emin olmak için).

Sarmalayıcılar, orta katman yazılımlarına (middleware) göre modellenmiştir – bunlar, argümanlarından biri olarak başka bir çağrılabilir işlevi alan çağrılabilir işlevlerdir. Bu çağrılabilir işlevi çağırarak (muhtemelen sarmalanmış) veritabanı sorgusunu çalıştırırlar ve bu çağrı etrafında istedikleri işlemleri yapabilirler. Ancak bunlar, kullanıcı kodu tarafından oluşturulur ve kurulur; bu nedenle, orta katman yazılımlarının yaptığı gibi ayrı bir fabrikaya ihtiyaç duymazlar.

Bir sarmalayıcı, bağlam yöneticisi içinde kurulur; bu nedenle sarmalayıcılar geçicidir ve kodunuzdaki belirli bir akışa özeldir.

Yukarıda belirtildiği gibi, bir sarmalayıcı örneği sorgu yürütme engelleyicisidir. Şu şekilde görünebilir:

```
def blocker(*args):
    raise Exception("Burada veritabanına erişim izni yok.")
```

Ve bu, şablondan gelen sorguları engellemek için bir görünümde şu şekilde kullanılır:

```
from django.db import connection
from django.shortcuts import render


def my_view(request):
    context = {...}        # Tüm verileri içeren bağlamı oluşturmak için kod.
    template_name = ...
    with connection.execute_wrapper(blocker):
        return render(request, template_name, context)
```

Sarmalayıcılara gönderilen parametreler şunlardır:

- `execute` – sorguyu yürütmek için diğer parametrelerle birlikte çağrılması gereken bir çağrılabilir.
- `sql` – veritabanına gönderilecek SQL sorgusu olan bir `str`.
- `params` – SQL komutu için parametre değerlerinin bir listesi/tuple'ı veya sarmalanmış çağrı `executemany()` ise listelerin/tuple'ların bir listesi/tuple'ı.
- `many` – nihai olarak çağrılan çağrının `execute()` mı yoksa `executemany()` mı olduğunu (ve `params`'ın bir değer dizisi mi yoksa değer dizilerinin bir dizisi mi olması beklendiğini) belirten bir `bool`.
- `context` – çağrı bağlamı hakkında daha fazla veri içeren bir sözlük. Buna bağlantı ve imleç de dahildir.

Parametreleri kullanarak, engelleyicinin biraz daha karmaşık bir versiyonu hata mesajına bağlantı adını ekleyebilir:

```
def blocker(execute, sql, params, many, context): alias =
    context["connection"].alias
    raise Exception("'{}' veritabanına erişim burada engellendi".format(alias))
```

Daha kapsamlı bir örnek olarak, bir sorgu günlüğü şunun gibi görünebilir:

```
import time
```

(devamı sonraki sayfada)

(önceki sayfadan devam)

```
class QueryLogger:
    def __init__(self):
        self.queries = []

    def __call__(self, execute, sql, params, many, context): current_query =
        {"sql": sql, "params": params, "many": many} start = time.monotonic()
        dene:
            sonuç = execute(sql, params, many, context)
        except Exception as e:
            current_query["status"] = "error"
            current_query["exception"] = e raise
        else:
            current_query["status"] = "ok"
            sonuç döndür
        son olarak:
            süre = time.monotonic() - başlangıç
            current_query["süre"] = süre
            self.sorgular.append(current_query)
```

Bunu kullanmak için bir logger nesnesi oluşturup bunu bir sarmalayıcı olarak yüklemelisiniz:

```
from django.db import connection

ql = QueryLogger()
with connection.execute_wrapper(ql):
    do_queries()
# Artık günlüğü yazdırabiliriz.
print(ql.queries)
```

connection.execute_wrapper()

execute_wrapper(wrapper)

Giriş yapıldığında veritabanı sorgu yürütmelerinin etrafına bir sarmalayıcı yerleştiren ve çıkış yapıldığında sarmalayıcıyı kaldıran bir bağlam yöneticisi döndürür. Sarmalayıcı, iş parçacığına özgü bağlantı nesnesine yerleştirilir.

`wrapper`, beş argüman alan bir çağrılabilir işlevdir. Bu işlev, bağlam yöneticisinin kapsamındaki her sorgu yürütülüşünde, yukarıda açıklandığı gibi `execute`, `sql`, `params`, `many` ve `context` argümanlarıyla çağrılır. İşlevin `execute(sql, params, many, context)` iş levini çağırması ve bu çağrının dönüş değerini döndürmesi beklenir.

3.2.12 Fikstürler

Bir fixture, veritabanının serileştirilmiş içeriğini içeren bir dosya koleksiyonudur. Her fixture'ın benzersiz bir adı vardır ve fixture'ı oluşturan dosyalar, birden fazla uygulamada birden fazla dizine dağıtılabilir.

Ayrıca bakınız

- Modeller için başlangıç verileri nasıl sağlanır

Fikstür nasıl oluşturulur

Fikstürler, *manage.py dumpdata* komutu ile oluşturulabilir. Ayrıca, serileştirme araçlarını doğrudan kullanarak veya hatta elle yazarak özel fikstürler oluşturmak da mümkündür.

Fikstür nasıl kullanılır

Fikstürler, testler için veritabanını önceden doldurmak amacıyla kullanılabilir:

```
class MyTestCase(TestCase):
    fixtures = ["fixture-label"]
```

ya da *loaddata* komutunu kullanarak bazı başlangıç verileri girmek için:

```
django-admin loaddata <fixture etiketi>
```

Fikstürlerin nasıl keşfedildiği

Django, fiksürleri şu konumlarda arar:

1. Yüklü her uygulamanın `fixtures` dizininde
2. *FIXTURE_DIRS* ayarında listelenen herhangi bir dizinde
3. Fikstürün adını taşıyan tam yol

Django, bu konumlarda bulduğu ve verilen fixture adlarıyla eşleşen tüm fixture'ları yükleyecektir. Adı belirtilen fixture'ın bir dosya uzantısı varsa, yalnızca o türdeki fixture'lar yüklenecektir. Örneğin:

```
django-admin loaddata mydata.json
```

komutu yalnızca `mydata` adlı JSON fikstürlerini yükler. Fikstür uzantısı, bir serileştiricinin kayıtlı adıyla (ör. `json` veya `xml`) eşleşmelidir.

Uzantıları atlarsanız, Django mevcut tüm fixture türleri arasında eşleşen bir fixture arar. Örneğin:

```
django-admin loaddata mydata
```

komutları, `mydata` adlı herhangi bir fikstür türündeki fikstürü arar. Bir fikstür dizini `mydata.json` dosyasını içeriyorsa, bu fikstür bir JSON fikstürü olarak yüklenir.

Adlandırılan fikstürler dizin bileşenleri içerebilir. Bu dizinler arama yoluna dahil edilir. Örneğin:

```
django-admin loaddata foo/bar/mydata.json
```

komutu, yüklü her uygulama için `<app_label>/fixtures/foo/bar/mydata.json` dosyasını, *FIXTURE_DIRS* içindeki her dizin için `<dirname>/foo/bar/mydata.json` dosyasını ve `foo/bar/mydata.json` dosyasını arar.

Fixture yükleme sırası

Aynı çağrıda birden fazla fixture belirtilebilir. Örneğin:

```
django-admin loaddata memeliler kuşlar böcekler
```

veya bir test senaryosu sınıfında:

```
class AnimalTestCase(TestCase):
    fixtures = ["mammals", "birds", "insects"]
```

Fixture'ların yüklenme sırası, yönetim komutu kullanılırken ya da yukarıda gösterildiği gibi test senaryosu sınıfında listelenirken, bunların listelendiği sırayı takip eder.

Bu örneklerde, tüm uygulamalardan (*INSTALLED_APPS* içinde uygulamaların tanımlandığı sırayla) `mammals` adlı tüm fikstürler ilk olarak yüklenecektir. Ardından, tüm `birds` fikstürleri yüklenecek, bunu tüm `insects` fikstürleri takip edecektir.

Veritabanı arka ucu satır düzeyinde kısıtlamaları destekliyorsa, bu kısıtlamaların işlemin sonunda kontrol edileceğini unutmayın. Veritabanı yapılandırması ertelenmiş kısıtlama kontrolünü desteklemiyorsa, fikstürler arasındaki herhangi bir ilişki yükleme hatasına neden olabilir (örnek için MySQL belgelerine bakın).

Fikstürlerin veritabanına nasıl kaydedildiği

Fikstür dosyaları işlendiğinde, veriler olduğu gibi veritabanına kaydedilir. Model tarafından tanımlanan *save()* yöntemleri çağrılmaz ve *pre_save* veya *post_save* sinyalleri, örnek yalnızca modele özgü öznitelikler içerdiğinden `raw=True` ile çağrılır. Örneğin, fikstür yüklenirken mevcut olmayan ve aksi takdirde bir istisna oluşturacak ilişkili alanlara erişen işleyicileri devre dışı bırakmak isteyebilirsiniz:

```
from django.db.models.signals import post_save
from .models import MyModel
```

(sonraki sayfada devam ediyor)

(önceki sayfadan devam)

```
def my_handler(**kwargs):
    # fixture yüklenirken işleyiciyi devre dışı bırak
    if kwargs["raw"]:
        return
    ...


post_save.connect(my_handler,  sender=MyModel)
```

Bu mantığı kapsüllemek için bir dekoratör de yazabilirsiniz:

```
from functools import wraps


def disable_for_loaddata(signal_handler): """
    Fikstür verilerini yüklerken sinyal işleyicilerini devre dışı bırakan dekoratör.
    """

    @wraps(signal_handler)
    def wrapper(*args, **kwargs): if
        kwargs["raw"]:
            return
        signal_handler(*args, **kwargs)

    return wrapper





@disable_for_loaddata
def my_handler(**kwargs): ...
```

Bu mantığın, sadece *loaddata* sırasında değil, fikstürler deserialize edildiğinde de sinyalleri devre dışı bırakacağını unutmayın.

Sıkıştırılmış fikstürler

Fikstürler `zip`, `gz`, `bz2`, `lzma` veya `xz` formatında sıkıştırılabilir. Örneğin:

```
django-admin loaddata mydata.json
```

`mydata.json`, `mydata.json.zip`, `mydata.json.gz`, `mydata.json.bz2`, `mydata.json`.

`lzma` veya `mydata.json.xz` dosyalarından herhangi birini arar. Sıkıştırılmış arşivde bulunan ilk dosya kullanılır.

Aynı ada sahip ancak farklı türde iki fixtür bulunursa (örneğin, aynı fixtür dizininde `mydata.json` ve `mydata.xml.gz` bulunursa), fixtür yüklemesi iptal edilir ve *loaddata* çağrısında yüklenen tüm veriler veritabanından kaldırılır.

> **MyISAM ve fikstürler içeren MySQL**
>
> MySQL'in MyISAM depolama motoru işlemleri veya kısıtlamaları desteklemez; bu nedenle MyISAM kullanırsanız, hazırlık verilerinin doğrulanması sağlanmaz ve birden fazla işlem dosyası bulunursa geri alma işlemi gerçekleştirilmez.

Veritabanına özgü sabit veriler

Çoklu veritabanı kurulumunda iseniz, bir veritabanına yüklemek istediğiniz, ancak diğerine yüklemek istemediğiniz sabit verileriniz olabilir. Bu durumda, sabitlerinizin adlarına bir veritabanı tanımlayıcısı ekleyebilirsiniz.

Örneğin, *DATABASES* ayarınızda bir `users` veritabanı tanımlanmışsa, fikstüre `mydata.users.json` veya `mydata.users.json.gz` adını verin; böylece fikstür, yalnızca `users` veritabanına veri yüklemek istediğinizi belirttiğinizde yüklenecektir.

3.2.13 Model ilişkisi API kullanımına örnekler

Çoktan çoka ilişkiler

Çok-çok ilişkisini tanımlamak için *ManyToManyField'ı* kullanın.

Bu örnekte, bir `Article` nesnesi birden fazla `Publication` nesnesinde yayınlanabilir ve bir `Publication` nesnesi birden fazla `Article` nesnesine sahip olabilir:

```
from django.db import models


class Publication(models.Model):
    title = models.CharField(max_length=30)

    class Meta:
        sıralama = ["başlık"]

    def __str__(self): return
        self.title


sınıf Makale(modeller.Model):
    başlık = models.CharField(max_length=100)
```

(sonraki sayfada devam ediyor)

(önceki sayfadan devam)

```
    yayınlar = models.ManyToManyField(Publication)

    sınıf Meta:
        sıralama = ["başlık"]

    def __str__(self): return
        self.headline
```

Aşağıda, Python API özellikleri kullanılarak gerçekleştirilebilecek işlemlerin örnekleri verilmiştir. Birkaç `Publication` örneği oluşturun:

```
>>> p1 = Publication(title="The Python Journal")
>>> p1.save()
>>> p2 = Publication(title="Science News")
>>> p2.save()
>>> p3 = Publication(title="Science Weekly")
>>> p3.save()
```

`Makale` Oluşturma:

```
>>> a1 = Article(headline="Django ile web uygulamalarını kolayca oluşturabilirsiniz")
```

Kaydedilene kadar bunu bir `Yayınla` ilişkilendiremezsiniz:

```
>>> a1.publications.add(p1) Traceback
(en son çağrı en son):
---
ValueError: "<Article: Django, web uygulamalarını kolayca oluşturmanıza olanak tanır>" için␣
↪"id" alanı için bir değer girilmelidir.
```

Kaydedin!

```
>>> a1.save()
```

Makaleyi bir `Yayınla` `ilişkilendirin`:

```
>>> a1.publications.add(p1)
```

Başka bir `Makale` oluşturun ve yayınlarında görünmesini sağlayın:

```
>>> a2 = Article(headline="NASA Python kullanıyor")
>>> a2.save()
```

(sonraki sayfada devam ediyor)

(önceki sayfadan devam)

```
>>> a2.publications.add(p1, p2)
>>> a2.publications.add(p3)
```

İkinci kez eklemek sorun değildir, ilişki yinelenmez:

```
>>> a2.publications.add(p3)
```

Yanlış türde bir nesne eklenirse `TypeError hatası` verilir:

```
>>> a2.publications.add(a1) Traceback
(en son çağrı en son):
---
TypeError: 'Publication' nesnesi bekleniyor
```

create() işlevini kullanarak tek adımda bir `Yayın` oluşturun ve bir `Makaleye` ekleyin:

```
>>> new_publication = a2.publications.create(title="Highlights for Children")
```

`Makale` nesneleri, ilgili `Yayın` nesnelerine erişebilir:

```
>>> a1.publications.all()
<QuerySet [<Publication: The Python Journal>]>
>>> a2.publications.all()
<QuerySet [<Yayın: Highlights for Children>, <Yayın: Science News>,
↪<Yayın: Science Weekly>, <Yayın: The Python Journal>]>
```

`Yayın` nesneleri, ilgili `Makale` nesnelerine erişebilir:

```
>>> p2.article_set.all()
<QuerySet [<Article: NASA uses Python>]>
>>> p1.article_set.all()
<QuerySet [<Makale: Django ile web uygulamalarını kolayca oluşturabilirsiniz>, <Makale: NASA Python
kullanıyor>
↪]>
>>> Publication.objects.get(id=4).article_set.all()
<QuerySet [<Article: NASA Python kullanıyor>]>
```

Çok-çok ilişkileri, ilişkiler arası aramalar kullanılarak sorgulanabilir:

```
>>> Article.objects.filter(publications__id=1)
<QuerySet [<Article: Django ile web uygulamalarını kolayca oluşturabilirsiniz>, <Article: NASA Python
kullanıyor>
↪]>
>>> Article.objects.filter(publications__pk=1)
```

(sonraki sayfada devam ediyor)

(önceki sayfadan devam)

```
<QuerySet [<Makale: Django ile web uygulamalarını kolayca oluşturabilirsiniz>, <Makale: NASA Python
kullanıyor>
↪]>
>>> Article.objects.filter(publications=1)
<QuerySet [<Article: Django ile web uygulamalarını kolayca oluşturabilirsiniz>, <Article: NASA Python
kullanıyor>
↪]>
>>> Article.objects.filter(publications=p1)
<QuerySet [<Article: Django, web uygulamalarını kolayca oluşturmanıza olanak tanır>, <Article: NASA, Python
kullanıyor>
↪]>

>>> Article.objects.filter(publications__title__startswith="Science")
<QuerySet [<Article: NASA Python kullanıyor>, <Article: NASA Python kullanıyor>]>
```

```
>>> Article.objects.filter(publications__title__startswith="Science").distinct()
<QuerySet [<Article: NASA Python kullanıyor>]>
```

count() işlevi de *distinct()* işlevini dikkate alır:

```
>>> Article.objects.filter(publications title startswith="Science").count() 2

>>> Article.objects.filter(publications title startswith="Science").distinct().count() 1

>>> Article.objects.filter(publications__in=[1, 2]).distinct()
<QuerySet [<Makale: Django ile web uygulamalarını kolayca oluşturabilirsiniz>, <Makale: NASA Python
kullanıyor>
↪]>
>>> Article.objects.filter(publications_in=[p1, p2]).distinct()
<QuerySet [<Makale: Django ile web uygulamalarını kolayca oluşturabilirsiniz>, <Makale: NASA Python
kullanıyor>
↪]>
```

Ters m2m sorguları desteklenir (yani, *ManyToManyField* içermeyen tablodan başlayarak):

```
>>> Publication.objects.filter(id=1)
<QuerySet [<Publication: The Python Journal>]>
>>> Publication.objects.filter(pk=1)
<QuerySet [<Publication: The Python Journal>]>

>>> Publication.objects.filter(article_headline__startswith="NASA")
<QuerySet [<Yayın: Highlights for Children>, <Yayın: Science News>,
↪<Publication: Science Weekly>, <Publication: The Python Journal>]>
```

(sonraki sayfada devam ediyor)

(önceki sayfadan devam)

```
>>> Publication.objects.filter(article__id=1)
<QuerySet [<Yayın: The Python Journal>]>
>>> Publication.objects.filter(article__pk=1)
<QuerySet [<Publication: The Python Journal>]>
>>> Publication.objects.filter(article=1)
<QuerySet [<Publication: The Python Journal>]>
>>> Publication.objects.filter(article=a1)
<QuerySet [<Publication: The Python Journal>]>

>>> Publication.objects.filter(article__in=[1, 2]).distinct()
<QuerySet [<Publication: Highlights for Children>, <Publication: Science News>,
↪<Publication: Science Weekly>, <Publication: The Python Journal>]>
>>> Publication.objects.filter(article__in=[a1, a2]).distinct()
<QuerySet [<Yayın: Highlights for Children>, <Yayın: Science News>,
↪<Publication: Science Weekly>, <Publication: The Python Journal>]>
```

İlgili bir öğeyi hariç tutmak da beklediğiniz gibi çalışır (ancak ilgili SQL biraz karmaşıktır):

```
>>> Article.objects.exclude(publications=p2)
<QuerySet [<Article: Django lets you build web apps easily>]>
```

Bir `Yayın` silinirse, ona bağlı `Makale` örnekleri bu yayına erişemez:

```
>>> p1.delete()
>>> Publication.objects.all()
<QuerySet [<Yayın: Çocuklar İçin Öne Çıkanlar>, <Yayın: Bilim Haberleri>,
↪<Publication: Science Weekly>]>
>>> a1 = Article.objects.get(pk=1)
>>> a1.publications.all()
<QuerySet []>
```

Bir `Makaleyi` silersek, ilgili `Yayın` örnekleri ona erişemez:

```
>>> a2.delete()
>>> Article.objects.all()
<QuerySet [<Article: Django, web uygulamalarını kolayca oluşturmanıza olanak tanır>]>
>>> p2.article_set.all()
<QuerySet []>
```

M2M'nin "diğer" ucu üzerinden ekleme:

```
>>> a4 = Article(headline="NASA, Dünya'da akıllı yaşam buldu")
>>> a4.kaydet()
>>> p2.article_set.add(a4)
>>> p2.article_set.all()
<QuerySet [<Makale: NASA, Dünya'da akıllı yaşam buldu>]>
>>> a4.publications.all()
<QuerySet [<Yayın: Bilim Haberleri>]>
```

Anahtar kelimeler kullanarak diğer uçtan ekleme:

```
>>> new_article = p2.article_set.create(headline="Oksijensiz diyet harikalar yaratıyor")
>>> p2.article_set.all()
<QuerySet [<Article: NASA, Dünya'da akıllı yaşam buldu>, <Article: Oksijensiz diyet␣
↪harikalar yaratıyor>]>
>>> a5 = p2.article_set.all()[1]
>>> a5.publications.all()
<QuerySet [<Yayın: Bilim Haberleri>]>
```

Bir Makaleden Yayın Silme:

```
>>> a4.publications.remove(p2)
>>> p2.article_set.all()
<QuerySet [<Article: Oksijensiz diyet harikalar yaratıyor>]>
>>> a4.publications.all()
<QuerySet []>
```

Ve diğer taraftan:

```
>>> p2.article_set.remove(a5)
>>> p2.article_set.all()
<QuerySet []>
>>> a5.publications.all()
<QuerySet []>
```

İlişki kümeleri şu şekilde ayarlanabilir:

```
>>> a4.publications.all()
<QuerySet [<Publication: Science News>]>
>>> a4.publications.set([p3])
>>> a4.publications.all()
<QuerySet [<Publication: Science Weekly>]>
```

İlişki kümeleri temizlenebilir:

```
>>>    p2.article_set.clear()
>>> p2.article_set.all()
<QuerySet []>
```

Ve diğer uçtan da temizleyebilirsiniz:

```
>>> p2.article_set.add(a4, a5)
>>>    p2.article_set.all()
<QuerySet [<Article: NASA, Dünya'da akıllı yaşam buldu>, <Article: Oksijensiz diyet␣
↪harikalar yaratıyor>]>
>>>    a4.publications.all()
<QuerySet [<Yayın: Science News>, <Yayın: Science Weekly>]>
>>>    a4.publications.clear()
>>>    a4.publications.all()
<QuerySet []>
>>>    p2.article_set.all()
<QuerySet [<Makale: Oksijensiz diyet harikalar yaratıyor>]>
```

Sildiğimiz `Makale` ve `Yayını` yeniden oluşturun:

```
>>> p1 = Publication(title="The Python Journal")
>>> p1.save()
>>> a2 = Article(headline="NASA uses Python")
>>> a2.save()
>>> a2.publications.add(p1, p2, p3)
```

Bazı `Publication` örneklerini toplu olarak silin; silinen yayınlara yapılan referanslar artık ilgili girdilerde yer almayacaktır:

```
>>> Publication.objects.filter(title__startswith="Science").delete()
>>>    Publication.objects.all()
<QuerySet [<Publication: Highlights for Children>, <Publication: The Python Journal>]>
>>>    Article.objects.all()
<QuerySet [<Makale: Django ile web uygulamalarını kolayca oluşturabilirsiniz>, <Makale: NASA,␣
↪ Dünya'da akıllı yaşam buldu>, <Article: NASA Python kullanıyor>, <Article: Oksijensiz diyet␣
↪harikalar yaratıyor>]>
>>>    a2.publications.all()
<QuerySet [<Publication: The Python Journal>]>
```

Bazı makaleleri toplu olarak sil - silinen nesnelere yapılan referanslar kaldırılmalıdır:

```
>>> q = Article.objects.filter(headline__startswith="Django")
>>> print(q)
<QuerySet [<Article: Django lets you build web apps easily>]>
>>> q.delete()
```

delete() işleminden sonra, *QuerySet* önbelleğinin temizlenmesi gerekir ve başvurulan nesneler silinmiş olmalıdır:

```
>>> print(q)
<QuerySet []>
>>> p1.article_set.all()
<QuerySet [<Article: NASA uses Python>]>
```

Çok-bir ilişkileri

Çok-bir ilişkisini tanımlamak için *ForeignKey* kullanın.

Bu örnekte, bir `Reporter` birçok `Article` nesnesiyle ilişkilendirilebilir, ancak bir `Article` yalnızca bir `Reporter` nesnesine sahip olabilir:

```
from django.db import models


class Reporter(models.Model):
    first_name = models.CharField(max_length=30) last_name
    = models.CharField(max_length=30) email =
    models.EmailField()

    def __str__(self):
        return f"{self.first_name} {self.last_name}"


class Article(models.Model):
    başlık = models.CharField(max_length=100) yayın_tarihi =
    models.DateField()
    reporter =models.ForeignKey(Reporter, on_delete=models.CASCADE)

    def __str__(self): return
        self.headline

    sınıf Meta:
        sıralama = ["başlık"]
```

Aşağıda, Python API özellikleri kullanılarak gerçekleştirilebilecek işlemlerin örnekleri verilmiştir.

Birkaç muhabir oluşturun:

```
>>> r = Reporter(first_name="John", last_name="Smith", email="john@example.com")
>>> r.save()

>>> r2 = Reporter(first_name="Paul", last_name="Jones", email="paul@example.com")
>>> r2.save()
```

Makale Oluşturma:

```
>>> from datetime import date
>>> a = Article(id=None, headline="Bu bir test", pub_date=date(2005, 7, 27),␣
↪reporter=r)
>>> a.save()

>>> a.reporter.id 1

>>> a.reporter
<Reporter: John Smith>
```

Bir nesneyi yabancı anahtar ilişkisine atayabilmek için önce onu kaydetmeniz gerektiğini unutmayın. Örneğin, kaydedilmemiş `bir Muhabir` ile bir `Makale` oluşturmak `ValueError hatasına` neden olur:

```
>>> r3 = Reporter(first_name="John", last_name="Smith", email="john@example.com")
>>> Article.objects.create(
...     headline="Bu bir test", pub_date=date(2005, 7, 27), reporter=r3
... )
Traceback (en son çağrı en son):
---
ValueError: kaydedilmemiş ilgili nesne nedeniyle veri kaybını önlemek için save() yasaklandı
↪'reporter'.
```

Makale nesneleri, ilişkili Reporter nesnelerine erişebilir:

```
>>> r = a.reporter
```

Reporter nesnesi aracılığıyla bir Article oluşturun:

```
>>> new_article = r.article_set.create(
...     başlık="John'un ikinci hikayesi", yayın_tarihi=tarih(2005, 7, 29)
... )
>>> new_article
```

(sonraki sayfada devam ediyor)

(önceki sayfadan devam)

```
<Makale: John'un ikinci hikayesi>
>>>   new_article.reporter
<Muhabir: John Smith>
>>> new_article.reporter.id 1
```

Yeni bir makale oluşturun:

```
>>> new_article2 = Article.objects.create(
...       başlık="Paul'un hikayesi", yayın_tarihi=tarih(2006, 1, 17), muhabir=r
... )
>>>   new_article2.reporter
<Reporter: John Smith>
>>> new_article2.reporter.id 1
>>>   r.article_set.all()
<Sorgu Kümesi [<Makale: John'un ikinci hikayesi>, <Makale: Paul'un hikayesi>, <Makale: Bu bir␣
↪test>]>
```

Aynı makaleyi farklı bir makale kümesine ekleyin - taşındığını kontrol edin:

```
>>>      r2.article_set.add(new_article2)
>>> new_article2.reporter.id 2
>>>   new_article2.reporter
<Muhabir: Paul Jones>
```

Yanlış türde bir nesne eklenirse TypeError hatası oluşur:

```
>>> r.article_set.add(r2) Traceback
(en son çağrı en sonda):
---
TypeError: 'Article' örneği bekleniyordu, <Reporter: Paul Jones> bulundu

>>>   r.article_set.all()
<QuerySet [<Article: John's second story>, <Article: This is a test>]>
>>>   r2.article_set.all()
<QuerySet [<Article: Paul'un hikayesi>]>

>>> r.article_set.count() 2
```

(sonraki sayfada devam ediyor)

(önceki sayfadan devam)

```
>>> r2.article_set.count() 1
```

Son örnekte makalenin John'dan Paul'a taşındığına dikkat edin.

İlgili yöneticiler de alan aramalarını destekler. API, ilişkileri istediğiniz kadar otomatik olarak takip eder. İlişkileri ayırmak için çift alt çizgi kullanın. Bu, istediğiniz kadar derinlikte çalışır. Herhangi bir sınırlama yoktur. Örneğin:

```
>>> r.article_set.filter(headline__startswith="This")
<QuerySet [<Article: This is a test>]>

# Adı "John" olan tüm muhabirlerin tüm makalelerini bul.
>>> Article.objects.filter(reporter__first_name="John")
<QuerySet [<Article: John'un ikinci hikayesi>, <Article: Bu bir test>]>
```

Burada tam eşleşme kastedilmektedir:

```
>>> Article.objects.filter(reporter__first_name="John")
<QuerySet [<Article: John's second story>, <Article: This is a test>]>
```

İlgili alan üzerinde iki kez sorgu yapın. Bu, WHERE cümlesinde bir AND koşuluna karşılık gelir:

```
>>> Article.objects.filter(reporter__first_name="John", reporter____last_name="Smith")
<QuerySet [<Article: John'un ikinci hikayesi>, <Article: Bu bir test>]>
```

İlişkili aramada bir birincil anahtar değeri girebilir veya ilişkili nesneyi açıkça aktarabilirsiniz:

```
>>> Article.objects.filter(reporter___pk=1)
<QuerySet [<Article: John's second story>, <Article: This is a test>]>
>>> Article.objects.filter(reporter=1)
<QuerySet [<Article: John's second story>, <Article: This is a test>]>
>>> Article.objects.filter(reporter=r)
<QuerySet [<Makale: John'un ikinci hikayesi>, <Makale: Bu bir test>]>

>>> Article.objects.filter(reporter__in=[1, 2]).distinct()
<QuerySet [<Makale: John'un ikinci hikayesi>, <Makale: Paul'un hikayesi>, <Makale: Bu bir␣
↪test>]>
>>> Article.objects.filter(reporter__in=[r, r2]).distinct()
<QuerySet [<Makale: John'un ikinci hikayesi>, <Makale: Paul'un hikayesi>, <Makale: Bu bir␣
↪test>]>
```

Örneklerin sabit listesi yerine bir sorgu kümesi de kullanabilirsiniz:

```
>>> Article.objects.filter(
...     reporter__in=Reporter.objects.filter(first_name="John")
... ).distinct()
<QuerySet [<Article: John'un ikinci hikayesi>, <Article: Bu bir test>]>
```

Ters yönde sorgulama:

```
>>> Reporter.objects.filter(article__pk=1)
<QuerySet [<Reporter: John Smith>]>
>>> Reporter.objects.filter(article=1)
<QuerySet [<Reporter: John Smith>]>
>>> Reporter.objects.filter(article=a)
<QuerySet [<Reporter: John Smith>]>

>>> Reporter.objects.filter(article__headline__startswith="This")
<QuerySet [<Reporter: John Smith>, <Reporter: John Smith>, <Reporter: John Smith>]>
>>> Reporter.objects.filter(article__headline__startswith="This").distinct()
<QuerySet [<Reporter: John Smith>]>
```

Ters yönde sayma, `distinct()` ile birlikte çalışır:

```
>>> Reporter.objects.filter(article__headline__startswith="This").count() 3
>>> Reporter.objects.filter(article__headline__startswith="This").distinct().count() 1
```

Sorgular döngüye girebilir:

```
>>> Reporter.objects.filter(article__reporter__first_name__startswith="John")
<QuerySet [<Reporter: John Smith>, <Reporter: John Smith>, <Reporter: John Smith>,
↪<Reporter: John Smith>]>
>>> Reporter.objects.filter(article__reporter__first_name__startswith="John").distinct()
<QuerySet [<Muhabir: John Smith>]>
>>> Reporter.objects.filter(article__reporter=r).distinct()
<QuerySet [<Reporter: John Smith>]>
```

Bir muhabiri silerseniz, makaleleri de silinecektir (ForeignKey'in *django.db.models.ForeignKey.on_delete* ayarı varsayılan olan `CASCADE` olarak tanımlandığını varsayarsak):

```
>>> Article.objects.all()
<QuerySet [<Article: John's second story>, <Article: Paul's story>, <Article: This is a␣
```

(devamı sonraki sayfada)

(önceki sayfadan devam)

```
↪test>]>
>>>     Reporter.objects.order_by("first_name")
<QuerySet [<Reporter: John Smith>, <Reporter: Paul Jones>]>
>>> r2.delete()
>>>    Article.objects.all()
<QuerySet [<Article: John's second story>, <Article: This is a test>]>
>>>     Reporter.objects.order_by("first_name")
<QuerySet [<Muhabir: John Smith>]>
```

Sorguda JOIN kullanarak silebilirsiniz:

```
>>> Reporter.objects.filter(article__headline__startswith="This").delete()
>>>   Reporter.objects.all()
<QuerySet []>
>>>    Article.objects.all()
<QuerySet []>
```

Bire bir ilişkiler

Bire bir ilişkisini tanımlamak için *OneToOneField* kullanın. Bu

örnekte, bir `Place` isteğe bağlı olarak bir `Restaurant` olabilir:

```
django.db'den modeller'i içe
aktar


class Place(models.Model):
    name = models.CharField(max_length=50)
    address = models.CharField(max_length=80)

    def __str__(self):
        return f"{self.name} the place"



class Restaurant(models.Model): place =
    models.OneToOneField(
        Place,
        on_delete=models.CASCADE,
        primary_key=True,
    )
    serves_hot_dogs = models.BooleanField(default=False)
```

(devamı sonraki sayfada)

(önceki sayfadan devam)

```
    serves_pizza = models.BooleanField(default=False)

    def __str__(self):
        return "%s restoranı" % self.place.name


class Garson(models.Model):
    restaurant = models.ForeignKey(Restaurant, on_delete=models.CASCADE) name =
    models.CharField(max_length=50)

    def __str__(self):
        return "%s'deki %s adlı garson" % (self.name, self.restaurant)
```

Aşağıda, Python API özellikleri kullanılarak gerçekleştirilebilecek işlemlerin örnekleri verilmiştir. Birkaç Place oluşturun:

```
>>> p1 = Place(name="Demon Dogs", address="944 W. Fullerton")
>>> p1.save()
>>> p2 = Place(name="Ace Hardware", address="1013 N. Ashland")
>>> p2.save()
```

Bir Restaurant oluşturun. Bu nesnenin birincil anahtarı olarak "parent" nesnesini geçirin:

```
>>> r = Restaurant(place=p1, serves_hot_dogs=True, serves_pizza=False)
>>> r.save()
```

Bir Restaurant, bulunduğu yere erişebilir:

```
>>> r.place
<Place: Demon Dogs the place>
```

Bir Mekan, varsa restoranına erişebilir:

```
>>> p1.restaurant
<Restoran: Demon Dogs restoran>
```

p2'nin ilişkili bir restoranı yoktur:

```
>>> from django.core.exceptions import ObjectDoesNotExist
>>> try:
...     p2.restaurant
```

(devamı sonraki sayfada)

(önceki sayfadan devam)

```
... ObjectDoesNotExist hariç:
...     print("Burada restoran yok.")
...
Burada restoran yok.
```

İstisna yakalama ihtiyacını ortadan kaldırmak için `hasattr`'ı da kullanabilirsiniz:

```
>>> hasattr(p2, "restaurant") False
```

Atama notasyonu kullanarak place değişkenini ayarlayın. place, Restaurant tablosundaki birincil anahtar olduğu için, bu kaydetme işlemi yeni bir restoran oluşturacaktır:

```
>>> r.place = p2
>>> r.save()
>>> p2.restaurant
<Restaurant: Ace Hardware the restaurant>
>>> r.place
<Place: Ace Hardware the place>
```

Ters yönde atama kullanarak yeri tekrar ayarlayın:

```
>>> p1.restaurant = r
>>> p1.restoran
<Restaurant: Demon Dogs restoranı>
```

Bir nesneyi bire bir ilişkisine atayabilmek için önce onu kaydetmeniz gerektiğini unutmayın. Örneğin, kaydedilmemiş `bir Place` ile bir `Restaurant` oluşturmak `ValueError hatası` verir:

```
>>> p3 = Place(name="Demon Dogs", address="944 W. Fullerton")
>>> Restaurant.objects.create(place=p3, serves_hot_dogs=True, serves_pizza=False) Traceback (en
son çağrı en son):
...
ValueError: Kaydedilmemiş 'place' nesnesi nedeniyle veri kaybını önlemek için save() işlevi engellendi.
```

Restaurant.objects.all() Places değil, Restaurants'ı döndürür. İki restoran olduğunu unutmayın - Ace Hardware the Restaurant, r.place = p2 çağrısında oluşturulmuştur:

```
>>>     Restaurant.objects.all()
<QuerySet [<Restaurant: Demon Dogs the restaurant>, <Restaurant: Ace Hardware the␣
↪restaurant>]>
```

Place.objects.all(), Restoran olup olmadığına bakılmaksızın tüm Yerleri döndürür:

```
>>> Place.objects.order_by("name")
<QuerySet [<Place: Ace Hardware the place>, <Place: Demon Dogs the place>]>
```

İlişkiler arasında arama yaparak modelleri sorgulayabilirsiniz:

```
>>> Restaurant.objects.get(place=p1)
<Restaurant: Demon Dogs restoran>
>>> Restaurant.objects.get(place_pk=1)
<Restaurant: Demon Dogs restoranı>
>>> Restaurant.objects.filter(place__name__startswith="Demon")
<QuerySet [<Restaurant: Demon Dogs the restaurant>]>
>>> Restaurant.objects.exclude(place__address__contains="Ashland")
<QuerySet [<Restaurant: Demon Dogs the restaurant>]>
```

Bu, tersi şekilde de çalışır:

```
>>> Place.objects.get(pk=1)
<Place: Demon Dogs the place>
>>> Place.objects.get(restaurant__place=p1)
<Place: Demon Dogs mekan>
>>> Place.objects.get(restaurant=r)
<Yer: Demon Dogs the place>
>>> Place.objects.get(restaurant_place__name__startswith="Demon")
<Place: Demon Dogs the place>
```

Bir yeri silerseniz, o yerdeki restoran da silinecektir (`OneToOneField`'ın *on_delete* ayarının varsayılan değer olan `CASCADE` olarak tanımlandığı varsayılırsa):

```
>>> p2.delete()
(2, {'one_to_one.Restaurant': 1, 'one_to_one.Place': 1})
>>> Restaurant.objects.all()
<QuerySet [<Restaurant: Demon Dogs the restaurant>]>
```

Restorana bir Garson ekleyin:

```
>>> w = r.waiter_set.create(name="Joe")
>>> w
<Garson: Demon Dogs restoranındaki Joe>
```

Garsonları sorgula:

```
>>> Waiter.objects.filter(restaurant__place=p1)
```

(devamı sonraki sayfada)

(önceki sayfadan devam)

```
<QuerySet [<Waiter: Demon Dogs restoranındaki garson Joe>]>
>>> Waiter.objects.filter(restaurant_place__name__ startswith="Demon")
<QuerySet [<Garson: Demon Dogs restoranında çalışan Joe>]>
```